COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE-010 - Data Structures and Algorithms	**SECTION:**	CPE21S2
DATE PERFORMED:	10/21/26	**DATE SUBMITTED:**	10/21/26
NAME:	SUBALISID, JAPPETH ROYCE B. BAUTISTA, MARK JONEIL S.	**INSTRUCTOR:**	ENGR. JIMLORD M. QUEJADO

Hands-on Activity 14.1	**Algorithm Complexity**

1. PROCEDURE OUTPUT

Tabulated below, are a number of functions against n (from 1 to 10)

A = (log2 n) {log to base 2 of n}
B = n {linear in n}
C = (n (log2 n)) {n log n}*
D = (n n) {quadratic in n}*
E = (n n n) {cubic in n}*
F = (expt 2 n) {exponential in n}
G = (expt 3 n) {exponential in n}
H = (fact n) {factorial in n}

n A B C D E F G H
1 0.0 1 0.0 1 1 2 3 1
2 1.0 2 2.0 4 8 4 9 2
3 1.0 3 4.0 9 27 8 27 6
4 2.0 4 8.0 16 64 16 81 24
5 2.0 5 11.0 25 125 32 243 120
6 2.0 6 15.0 36 216 64 729 720
7 2.0 7 19.0 49 343 128 2187 5040
8 3.0 8 24.0 64 512 256 6561 40320
9 3.0 9 28.0 81 729 512 19683 362880
10 3.0 10 33.0 100 1000 1024 59049 3628800

Think of this as algorithms A through H with complexities as defined above, showing growth rate versus input size n. Tabulated below are functions F, G and H from above (ie 2 to power n, 3 to power n, and n factorial). Problem size n varies from 10 to 100 in steps of 10. I have assumed that we have a machine that can perform "the principle activity of the algorithm" in a micro second (ie. if we are considering a graph colouring algorithm, it can compare the colour of two nodes in a millionth of a second). The columns give the number of years this machine would take to execute those algorithms on problems of size n (note: YEARS). This is expressed as 10^x, "10 raised to the power x".

n F G H
10 10^-10 10^-8 10^-6

20 10^-7 10^-3 10^4
30 10^-4 10^0 10^18
40 10^-1 10^5 10^34
50 10^1 10^10 10^50
60 10^4 10^15 10^68
70 10^7 10^19 10^86
80 10^10 10^24 10^105
90 10^13 10^29 10^124
100 10^16 10^34 10^144

Therefore, if we have a problem of size (lets say 40) and the machine specified above, if the best algorithm is O(2**n) it will take 1 year, if the best algorithm is O(3**n) it will take 100,000 years, and if the best algorithm is O(n!) it will take

10,000,000,000,000,000,000,000,000,000,000,000 years

approximately. Out of interest, the age of the universe is estimated to be between 15 and 20 billion years old, i.e. 20,000,000,000 years. That is, even at modest values of n we are presented with problems that will never be solved. It is almost tempting to say that from a computational perspective, the universe is a small thing.

Contrast and compare two algorithms used in sorting. For the two algorithms, identify the best and worst case. Additionally, provide an analysis of the algorithms if they were to sort data with the following values of n: 100, 1,000, and 10,000 (use experimental and theoretical analysis tools used in previous activities).

	Best Case	**Worst Case**	**Analysis**
Algorithm 1: Insertion Sort	O(n)	O(n^2)	Operating at O(n^2) worst case complexity, Insertion Sort requires n =100, 1,000,000 operations for n = 1,000, and 100,000,000 for n =10,000. This quadratic growth makes it highly inefficient for large unsorted datasets despite its linear best case efficiency
Algorithm 2: Shell Sort	O(n log n)	O(n^2)	Using a standard gap sequence with O(n^2) worst case complexity, Shell Sort takes approximately 1,000 operations for n = 100, 31, 623 operations for n = 1,000 and 1,000,000 operations for n =10,000.

			Comparing distant elements enables Shellsort to scale significantly better than Insertion Sort as n grows.

Table 14-1. Best- and Worst-Case Analysis using Theoretical Tools

	Algorithm 1: Insertion Sort	**Algorithm 2: Shell Sort**
Best Case: Already sorted	N = 100 800 nanoseconds (Insertion sort) N = 1,000 4600 nanoseconds (Insertion sort) N = 10,000 43700 nanoseconds (Insertion sort)	N = 100 2800 nanosecond(Shell sort) N = 1,000 38200 nanosecond(Shell sort) N = 10,000 575300 nanosecond(Shell sort)
Worst Case: Reverse order	N = 100 18100 nanoseconds (Insertion sort) N = 1,000 2528700 nanoseconds (Insertion sort) N = 10,000 157015500 nanoseconds (Insertion sort)	N = 100 5400 nanosecond(Shell sort) N = 1,000 58100 nanosecond(Shell sort) N = 10,000 778700 nanosecond(Shell sort)

Table 14-2. Best- and Worst-Case Analysis using Experimental Tools

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO A: Contrast and compare the two searching techniques used in class. The required deliverable for this activity would be theoretical and empirical analysis. For the theoretical analysis, the deliverable must be:

- Pseudocode Analysis
- Time and Space Complexities

Pseudocode

Linear Search:

```
Algorithm LinearSearch(array, N, target):
    for i from 0 to N -1 do:
        if array[i] == target
            return i
    return -1
```

Binary Search:

```
Algorithm BinarySearch(array, N, target):
    low = 0, high = N - 1
    while low <= high do:
        mid = (low + high) / 2
        if array[mid] == target then
            return mid
        else if array[mid] < target then
            low = mid + 1
        else
            high = mid - 1
    return -1
```

Time and Space Complexities

Linear Search: Linear Search has a best case time complexity of O(1) when the target element is located at the first index, a worst case time complexity of O(N) when the target is at the end of the array or absent, and an auxiliary space complexity of O(1) because it only uses a constant amount of memory for its loop counter

Binary Search: Binary Search on the other hand achieves a best case time complexity of O(1) when the target element is found at the initial middle index, a worst case time complexity of O(log N) because it repeatedly halves the search space and an auxiliary space complexity of O(1) in its iterative form since it only tracks pointer variables.

For the empirical analysis, the deliverable must be the filled-out table and your analysis.
Code Used:

```
int find = 354649514;
    int arr[max_size];
    int arr2[max_size];
    srand(time(0));
    for (int i = 0; i < max_size; i++) {
    arr[i] = rand() % 100000;
    arr2[i] = rand() % 100000;
    }

    std::thread linear(linearSearchAr<int>, max_size, find, arr);
    std::thread binary(binarySearchAr<int>, arr, max_size, find);
    linear.join();
    binary.join();
```

All trials used the same data to find, while the data were all rand % maxsize.

Input Size (N)	Elapsed Time for Binary Search (Worst-Case)	Elapsed Time for Linear Search (Worst-Case)
1,000	200 nanosecond(Binary)	3200 nanosecond(Linear)
10,000	600 nanosecond(Binary)	29400 nanosecond(Linear)
100,000	800 nanosecond(Binary)	297600 nanosecond(Linear)

3. CONCLUSION

Bautista: Through theoretical and empirical evaluation, this activity highlighted the direct impact of algorithm complexity on practical performance. For Sorting, Insertion Sort degraded rapidly to 157,015,500 ns in the worst case at n = 10,000 due to its O(n²) complexity whereas Shell Sort scaled far better at 778, 700 ns. Similarly in searching operations, Binary search's logarithmic growth (O(log N) drastically outperformed Linear Search, (O(N), completing in 800 ns compared to 297,600 ns at 100,000. Ultimately, these results confirm that selecting algorithms with lower complexity growth rates is vital for maintaining system efficiency as dataset sizes scale.

Subalisid: Theoretically in our procedure, shell sort should be faster than insertion in the best case scenario. However, in the experimental results, we found that insertion—at higher and higher N value—is at least 10 times faster than shell. This is mainly because of the amount of comparisons that the shell sort has to do compared to linear who only needs to do one pass. On the other hand, when the array is in reverse order(worst-case) we can see the significant advantage that the shell sort has. It finished sorting the array almost 20 times faster than insertion. Therefore, while theoretical might contradict the best-case scenario—due to other factors—we still proved the superiority of shell sort in terms of sorting speed.
In the supplementary, the execution time for the worst cases shows how binary search is ultimately faster than linear. Binary increased almost negligibly while linear grew the same amount as the size of the array.
All in all, this activity helped me realize the importance of comparing theoretical with experimental runtimes to prevent accidental mishaps resulting in inefficient programs.

4. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.